



2950 Niles Road, St. Joseph, MI 49085-9659, USA
269.429.0300 fax 269.429.3852 hq@asabe.org www.asabe.org

An ASABE Meeting Presentation

DOI: <https://doi.org/10.13031/aim.202100193>

Paper Number: 2100193

Apple fruit detection and counting based on deep learning and trunk tracking

Fangfang Gao¹, Tingyi Yang¹, Longsheng Fu^{1,2,3,4,*}

1. College of Mechanical and Electronic Engineering, Northwest A&F University, Yangling 712100, China;
2. Key Laboratory of Agricultural Internet of Things, Ministry of Agriculture and Rural Affairs, Yangling 712100, China;
3. Shaanxi Key Laboratory of Agricultural Information Perception and Intelligent Service, Yangling 712100, China;
4. Centre for Precision and Automated Agricultural Systems, Washington State University, Prosser, WA 99350, USA.

**Written for presentation at the
2021 ASABE Annual International Meeting
Sponsored by ASABE
Virtual and On Demand
July 12-16, 2021**

ABSTRACT. *Accurate estimation of apple yield is important for producers to make informed decisions when managing their orchards and be competitive when selling their products. Yield estimation is a key for predicting stock volumes, allocating needed labor, and planning harvesting operations. Manual visual yield estimation has traditionally been employed, resulting in inaccurate and misleading information. Although some studies have developed related algorithms based on machine vision to count fruits in tree images, they are difficult to apply in large orchards. The main goal of this study is to develop an automated video processing methodology to detect and count the apple fruits in modern vertical fruiting-wall architecture using deep learning techniques and object tracking. Existing researches using tracking technology to count fruits based on videos have all put the tracking target on the tracking fruit target itself. However, the small size and large number of video fruit targets have brought great obstacles to the realization of tracking technology. Therefore, according to the fact that all relatively stationary targets in the video have the same displacement trajectory, this study select tree trunk as the tracking target to obtain the displacement trajectories of all detected fruits. A detection model of apples and tree trunks was trained used YOLOv4-tiny network, which integrated with the tracking algorithm to develop a counting algorithm for video fruits. Combining the detected position information of the apple and the trunk with the displacement track obtained by tracking can establish the connection between the fruits in the video frame, thereby realizing the counting of fruits. The developed counting method was used to detect and count the images and videos in testing dataset, respectively. An AP of 99.35% for apple and trunk detection was obtained in this study, and over 90% apples were counted correctly using proposed method compared with manual counting results. These promising results demonstrate the potential of the present method to provide yield estimates for apple fruits or even other types of fruit.*

Keywords. *yield estimation; orchard video; object tracking; fruit position; reference displacement; fruit ID*

The authors are solely responsible for the content of this meeting presentation. The presentation does not necessarily reflect the official position of the American Society of Agricultural and Biological Engineers (ASABE), and its printing and distribution does not constitute an endorsement of views which may be expressed. Meeting presentations are not subject to the formal peer review process by ASABE editorial committees; therefore, they are not to be presented as refereed publications. Citation of this work should state that it is published by ASABE. EXAMPLE: Author's Last Name, Initials. 2019. Title of presentation. Paper No. ——. St. Joseph, MI: ASABE. For information about securing permission to reprint or reproduce a meeting presentation, please contact ASABE at www.asabe.org/Copyright (2950 Niles Road, St. Joseph, MI 49085-9659 USA).

Introduction

Knowledge of tree fruit crop yield such as apples can guide agronomic treatments, harvest scheduling making, and labor resource allocation. Yield estimation in an apple orchard is important for growers as it facilitates efficient utilization of resources and improves returns per unit area and time (Bargoti and Underwood, 2017; Koirala et al., 2019). Despite the importance, the estimation is generally carried out in a method of manual counting combined with the orchard agronomic conditions and historical fruit yield data due to the lack of a more automatic and precise fruit yield estimation method (Payne et al., 2014; Dorj et al., 2017; Apolo-Apolo et al., 2020). However, manual counting is a labor-intensive, expensive, and often destructive method, which becomes virtually infeasible to be considered an effective tool in the decision-making process. Therefore, developing an automatic and highly accurate apple counting method to estimate apple yield before harvest is the key to helping farmers make decisions.

Fruit detection is a precursor for growers to automatically count fruits for yield estimation and numerous precision farming. In recent years, a wide range of methodologies have been developed to detect fruits. Traditional methods such as K-means clustering, simple linear iterative clustering, circular Hough transform, directional gradient histogram, etc. rely on the color, shape, and texture characteristics of the fruit can only achieve fruit detection with poor robustness (Qureshi et al., 2017; Wang, 2018; Liu et al., 2019b). The deep learning method based on the convolutional neural network with advanced semantic expression ability has good robustness, because it can autonomously learn the shallow features and deep features of the target (Fu et al., 2018; Zhou et al., 2020; Song et al., 2021). The two-stage detection network including Faster R-CNN (Faster Region Convolutional Neural Networks) has high detection precision (Gené-Mola et al., 2019; Gao et al., 2020), while Fu et al. (2020a) and Kuznetsova et al. (2020) proved that YOLO (You Only Look Once), which belongs to the one-stage detection network, has a faster detection speed. Therefore, YOLOv4-tiny (Wang et al., 2020), released with YOLOv4 (Bochkovskiy et al., 2020) in 2020, is considered to have the potential to achieve high-precision detection at a faster detection speed.

Some work has been achieved to count the number of tree fruit crops based on fruit detection. Stein et al. (2016) used Faster R-CNN based on VGG16 to count the obtained images of a mango tree from multiple angles. Compared with the results based on manual counting, the error rate of a single fruit tree is only 1.36%. Bargoti and Underwood (2017) and Dorj et al. (2017) achieved a correlation coefficient of 0.826 and 0.93 between the fruit count results of the algorithm developed based on watershed segmentation and manual counting for apple images and citrus images, respectively. Koirala et al. (2019) developed a network 'MangoYOLO' based on YOLOv3 and YOLOv2-tiny to detect and count mango fruits by images, which achieved orchard fruit load estimates of between 4.6 and 15.2% of actual fruit numbers. Although these methods obtained good fruit counting results, most of these methods are based on fruit tree images, which is time-consuming, labor intensive, costly and difficult to implement for apple orchards. Video of multiple fruit trees, which can be easily obtained by a self-built cart, is a convenient alternative that can help quickly achieve fruit counting. However, to realize the fruit counting in the video, a method is needed to correlate the same fruits between consecutive video frames to avoid over-counting in addition to realizing the detection of fruits.

Tracking, a fundamental task in any video application requiring some degree of reasoning about objects of interest, can establish object correspondences between video frames. Given the position of any object of interest in the first frame of the video, the purpose of visual object tracking is to estimate its position in all subsequent frames with the highest possible accuracy (Q. Wang et al., 2019). Tracking is widely used in the fields of military aerospace, security surveillance and intelligent driving (Lee et al., 2019; Ngo et al., 2019; Wawrzyniak et al., 2019). In recent years, tracking algorithms based on discriminative correlation filter (DCF) have developed into the mainstream video tracking method due to relatively stable tracking performance and fast speed (Kalal et al., 2012; Danelljan et al., 2014). A CSR-DCF (channel spatial reliability-discriminative correlation filter) tracking algorithm have been proposed on recent tracking benchmark that could achieve state-of-the-art performance where channel spatial reliability concepts to DCF tracking (Lukežič et al., 2018). Farhodov et al. (2019) integrated Faster R-CNN with CSR-DCF algorithm to track pedestrian and vehicle and demonstrated outstanding real-time detection and tracking performance.

Existing researches on video-based fruit counting are mostly implemented by tracking all fruits detected in a video frame. Liu et al. (2019a) developed a mango fruit counting system based on the Kalman filter to track all fruits, and a R2 value related to the field count reaches 0.88. Z. Wang et al. (2019) used a Kalman filter-based tracker to track all mango fruits detected in the video using the 'MangoYOLO' network with a bias corrected Root Mean Square Error (RMSE) of 18.0 fruit/tree. Vasconez et al. (2020) used Gaussian estimation multi-object tracking algorithm developed by Milan et al. (2017) to track all apple fruits detected by Faster R-CNN with Inception V2 in the video, which reached a 93% of counting performance. Although the above research has good results for fruit counting, the algorithms need to track all the detected fruits, which cause the algorithm to take up a lot of computing resources. Therefore, realizing fruit video counting through resource-saving single-target tracking may be more in line with actual application requirements.

By tracking the trunk of apple tree trained in vertical fruiting-wall architecture, the motion trajectories of all relatively stationary objects in the video frame, including trunks, branches, leaves and fruits, can be obtained. The existing video tracking technology mainly focuses on tracking some large targets with obvious characteristics, such as pedestrians (Wojke

et al., 2018), cars (Li et al., 2014), or humans with actions (Jain et al., 2014), and has not reached the accuracy of human marking after decades of research (Milan et al., 2017). Therefore, it is difficult to track one of the apple fruits that have similar characteristics to each other in an unrestricted environment. However, the trunk target is larger and has the same trajectory as the fruit, which is more suitable for tracking than the fruit target. In the modern vertical fruiting-wall architecture, the trees have obvious characteristics and are planted close together with interplant spacing generally ranging from 0.3 to 1.5 m, while inter-row spacing varies typically from 2.5 to 4.0 m (Gao et al., 2020). Therefore, the relationship between the fruits of the upper and lower frames of the video can be established by tracking the tree trunk, thereby eliminating the repeated counting of fruits.

This study aims to design and implement a real-time apple video automatic counting system based on YOLOv4-tiny detection method and single-target tracking method for estimating the yield of apple orchards. This work is organized as follows: Section 2 mainly describes the design of the software algorithm to detect and count apples from the videos. Results and discussion for fruit detection on images, object tracking and counting for frame sequences, and future work are presented in Section 3, and the Summary and Conclusions in Section 4.

Materials and methods

This study focused on counting apples in vertical fruiting-wall tree architecture, which is one of the modern orchard systems. This section describes the self-built portable device that was used to acquire sensor data, the apple orchard where experiments were performed and the algorithmic pipeline that is proposed to process the data to detect, track, and count the fruits and trees. An overview of the pipeline is illustrated in Figure 1. Fruits and tree trunks are detected in individual color images and videos using YOLOv4-tiny, a state-of-the-art object detection network. The detection result of the tree trunk can be used as the input of the target tracking network to realize the association between the same targets in the video frame. Fruit counting algorithm was developed by integrating detection and tracking algorithms, which then enables the unique count. This section describes the equipment and all of the steps in Figure 1 in detail.

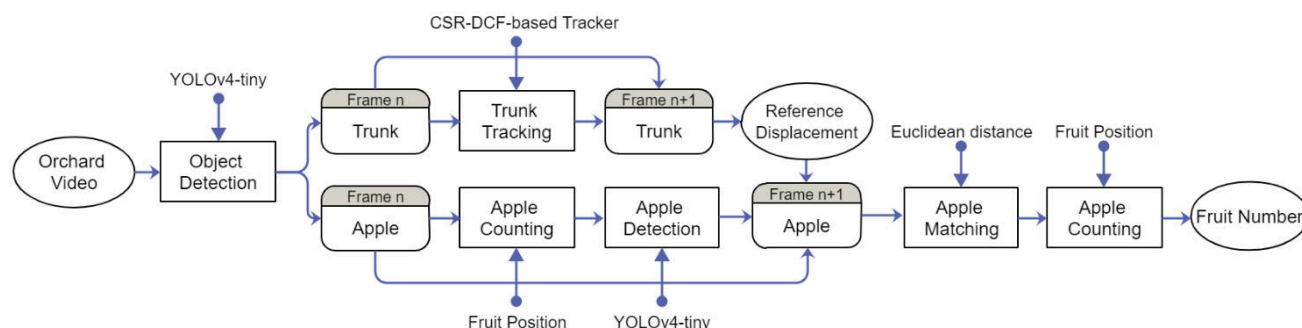


Figure 1. Proposed pipeline. To count fruits on one side of a specific row of trees, our pipeline takes in videos recorded from one side of the tree row. Firstly, fruits and tree trunks are detected using YOLOv4-tiny in frame n. The trunk detection result from the YOLOv4-tiny detector is input into the CSR-DCF-based tracker to track the trunk in subsequent video frames. Calculate the movement displacement of the tree trunk between consecutive video frames to match the same fruit. Based on the detected fruit positions, fruit IDs are given in order to achieve fruit counting, and the maximum ID of the fruit in the last frame of the video is obtained as the number of fruits in the video.

Data acquisition

Image and video data were collected using a self-built portable device in a commercial fruiting-wall Scifresh apple orchard with dense-foliage canopy. A Microsoft Kinect V2 sensor (Microsoft Inc., Redmond, WA, USA), a camera clamping device, and a support frame were mounted on a portable cart to form the device for acquiring fruit-on-plant images, while a field remote control trolley was used to get orchard videos. All image and video data in this study were acquired in a commercial, vertically trellis-trained apple orchard (Figure 2a, 2b) located in north Prosser, WA, USA. The tree spacing and row spacing are 1.5 m and 2.7 m, respectively, and tree height is about 4.0 m (Zhang et al., 2018; Fu et al., 2020b). In the orchard, regular canopy management is conducted annually and equally to all blocks and it is normally maintained by the same group of trained laborers.

A total of 800 RGB (Red, Green, and Blue) images and 10 videos were acquired during two harvesting seasons (2017 and 2018). All the apple images were obtained using the Kinect V2 sensor with a JPEG image format and a resolution of 1920×1080 pixels for developing an algorithm for fruit and trunk detection. During the apple videos acquisition, the trolley was driven along the rows of the apple tree and is tried to maintain at an almost constant speed. The time of video acquisition was random, so the numbers of fruit trees and fruits contained in the video were also different. The total numbers of fruit and fruit trees in each video were counted manually. Every fruit in each video including tree origin (where adjacent canopies inter-meshed) was counted, except for the fruits in the background. Uncertainty in manual count typically occurred around dimly lit fruit on the canopy margin and background and around highly occluded fruit (Figure 2c) (Qureshi et al., 2017). The RMSE for manual counts of the video set (mean of 43 fruit per image) by three different operators was 2.7 fruits per

video.



Figure 2. (a) Experimental orchard used in this work; and (b) Apple trees of the 'Scifresh' variety during harvest season.

Dataset building

The dataset is constructed using image data and video data to train and test network performance. The overall image data was randomly divided into raw training dataset (80% of the images) and testing dataset (20% of the images) to train and test the detection network, respectively. For each image, an XML annotation file (filename=image name) was created containing the image attributes (name, width, height) and the object attributes (class name, object bounding box co-ordinates). Apples and tree trunks were labeled as two different classes with rectangular annotations in the XML file. While all the videos were used to test the performance of the counting algorithm.

Data augmentation technique was used to enlarge the image dataset which helped to reduce over-fitting in supervised learning algorithms. To achieve sensitive detection of apple in the orchard, this study took into consideration most kinds of interference that may occur when detecting fruits. Data augmentation, including brightness transformation, adaptive histogram equalization, motion blur transformation and image mirroring in horizontal axis on the dataset, were implemented. A total of 800 images were augmented to 6400 images. Although many studies have proved that rotation can greatly improve network performance, considering that the angle of the fruit trees in the image generated by rotation does not conform to the actual situation, this study did not choose rotation as the augmented method.

YOLOv4-tiny detection network

In terms of fruit detection using CNN-based network, YOLOv4-tiny can be used to develop a network model that is faster than other methods. YOLO was first proposed in 2016 (Redmon et al., 2016) and constantly improved recent year giving rise to YOLOv2 (Redmon and Farhadi, 2017), YOLOv3 (Redmon and Farhadi, 2018), and YOLOv4 (Bochkovskiy et al., 2020). Corresponding lightweight networks, such as YOLOv3-tiny and YOLOv4-tiny have also been proposed. YOLOv4-tiny, a network scaled down by the YOLOv4 object detection neural network, is designed for low-end GPU devices and achieved extremely high 1774 FPS speed on RTX 2080Ti by using TensorRT-FP16 (Wang et al., 2020). YOLOv4-tiny achieved a 40.2% mAP@0.5, which is 7.1% higher than YOLOv3-tiny in the MS COCO dataset (Han et al., 2020). Nevertheless, the processing speed of YOLOv4-tiny is slightly faster than that of YOLOv3-tiny.

Structure of YOLOv4-tiny is shown in Figure 3. YOLOv4-tiny had fewer layers of 37 (starting from 0) compared to YOLOv4 with more than a hundred. Therefore, it made the training process faster with fewer computations without too much trade in detection performance (Montalbo, 2020). The extractor begins with an input layer that takes an $N \times N$ image, which is 416×416 in this paper. The backbone comes with an interchanging 3×3 and 1×1 receptive filters that strides over the input to produce a new set of feature maps or filters that pass through the network. Detector uses both the feature map of the final level and the feature map of the previous level to detect various size objects by applying the concept of feature pyramid network. Because many convolutional layers cannot be used on lower-level feature maps, it is difficult to extract enough feature maps. Therefore, YOLOv4-tiny uses only the two feature maps for object detection, unlike the YOLOv4.

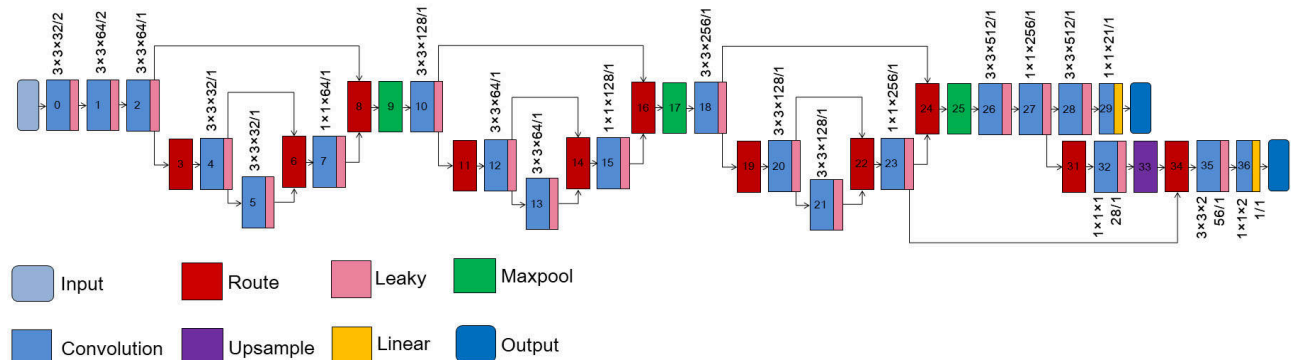


Figure 3. Structure of YOLOv4-tiny. The number of each layer in the network represents the number of layers of the network; The output feature map size of each layer in the network is expressed as 'width × height × number of filters/stride'.

CSR-DCF-based tracker

Object tracking is an open and challenging problem for which discriminative correlation filter have shown excellent performance. The basis of the proposed tracking method is based on the CSR-DCF algorithm, which has achieved the latest results on VOT 2016, VOT 2015 and OTB100, and can be run in real-time on a CPU (Lukežič et al., 2018). CSR-DCF performed on par with the VOT2016 best-performing CCOT (Danelljan et al., 2016), while being 20 times faster than the CCOT. The speed of real-time trackers like DSST (Danelljan et al., 2014) and Struck (Hare et al., 2011) is comparable to CSR-DCF, but their tracking performance is significantly poorer. KCF (Henriques et al., 2015) can run nearly 10 times faster than CSR-DCF, but delivers a significantly poorer performance than CSR-DCF, with an average failures of 2.03, while CSR-DCF is only 0.85.

The CSR-DCF algorithm has been implemented in C++ and integrated into the Open CV library as a DNN (Deep Neural Network) module (Farhodov et al., 2019). The tracker trains the DCF template based on two simple standard features, HoGs and Colornames, so as to realize the tracking of the target in the video sequence. In the tracking process, the filter is adjusted to the part of the object suitable for tracking by establishing a spatial reliability map, which can not only enlarge the search region, but also improve tracking of non-rectangular objects. Reliability scores reflect channel-wise quality of the learned filters and are used as feature weighting coefficients in localization, thereby reducing the noise of the weight-averaged filter response. The area where the maximum response point is the new position of the target, and then the predict area is used to train and update the filter for subsequent prediction.

Fruit counting algorithm

This research matched the same fruit between video frames by tracking tree trunk targets in the field of view. The YOLOv4-tiny network model trained in Section 2.3 can detect orchard video obtained in Section 2.1, thereby finding the number and position of the fruit and tree trunk in each frame. The fruit and the trunk were distinguished based on the class tag index, and the first trunk target in the video motion direction was selected as the tracking target. Initialize the tracker using the detected position of the tracking target to get the predicted position of the target in the next frame. Calculate the difference between the predicted position and the initial position to obtain the reference displacement between video frames. Because the displacement between video frames is not large, the new position obtained by adding the reference displacement to the initial position of most apples should ideally be the same as the position of a certain fruit detected in the next frame, thereby realizing the matching of the same fruit between consecutive frames of video.

The existence of detection error and tracking error makes it difficult to match the same fruit between consecutive frames of the video. Although both the YOLOv4-tiny and CSRT networks have good performance, there is a certain gap between the positions of the fruits output by the two networks and ground truth. However, there should be the closest Euclidean distance between the calculated new position of the fruit and the detected new position of the same fruit, so that preliminary matching of the same fruit can be achieved. Video quality problems or the loss of the tracking target field of view will cause tracking errors or even tracking failures. In order to avoid tracking errors resulting in large errors in the calculation of the reference displacement, the study will promptly adjust the position of the tracked tree trunk; at the same time, in order to avoid the inability to obtain the reference displacement due to the tracking failure, the study will also change the tracking target in advance.

Each frame of fruit detection results and tree trunk tracking results were used as input to the fruit counting algorithm. The position of the target obtained by detection and tracking is expressed in pixel coordinates (x, y, w, h) , where x, y respectively refer to the horizontal and vertical coordinates of the upper left corner of the target detection rectangle, w is the width of the rectangle, h is the height of the rectangular box. Let B_a^m be the bounding boxes of apple for frame m , B_t^m be the bounding boxes of trunk for frame m . Let (x', y') be the coordinates of the center point of the bounding box, which can be calculate in Eq. (1).

$$(x', y') = (x + \frac{w}{2}, y + \frac{h}{2}) \quad (1)$$

For frame 1, the algorithm will perform the following steps:

- (1) Detect video frame by YOLOv4-tiny to obtain B_a^1 and B_t^1 ;
- (2) Assign fruit IDs incrementally starting from 1 according to the video motion direction, store the current count to the list L , and obtain the current fruit number N ;
- (3) Extract the bounding box B_{ti}^1 of the trunk i with the smallest abscissa to initialize the tracker;

For frame 2:

- (1) Detect video frame by YOLOv4-tiny to obtain B_a^2 and B_t^2 ;
- (2) Assign fruit IDs incrementally starting from $N+1$ according to the video motion direction, store the current count to the list L , and obtain the current fruit number N ;
- (3) Extract the new bounding box coordinates B_{ti}' of the trunk i output by the tracker;
- (4) Calculate the overlap rate 'Op' between B_{ti}' and all tree trunk detection bounding boxes B_t^2 based on Eq. (2), if the maximum overlap is less than 30% (We experimented with [10%, 20%, 30%, 40%, and 50%] overlap threshold. We found 30% to be the most appropriate threshold for detecting overlaps.), which means that there is a big mistake in tracking, and then use the B_t^2 with the largest overlap rate to replace B_{ti}' to update the tracker;
- (5) Calculate the video frame reference displacement 's' based on Eq. (3);
- (6) Calculate the Euclidean distance 'Ed' between fruits detected in consecutive video frames based on Eq. (3), and the two fruits with the smallest Euclidean distance are the same fruit;
- (7) Change the ID of the same fruit in the current frame to the ID of the previous frame, update the list L and the number of current fruits N ;

$$Op_{B_{ti}^m}^{B_{ti}^m} = \frac{area(B_{ti} \cap B_t^m)}{area(B_{ti} \cup B_t^m)} \quad (2)$$

$$S(x_s, y_s) = B_{ti}(x_{B_{ti}}', y_{B_{ti}}') - B_t^{m-1}(x_{B_t^{m-1}}', y_{B_t^{m-1}}') \quad (3)$$

$$Ed = \sqrt{(x_{B_t^{m-1}+s}' - x_{B_t^m}')^2 + (y_{B_t^{m-1}+s}' - y_{B_t^m}')^2} \quad (4)$$

Repeat the steps of frame 2 till the field of view of current tracking trunk i is no longer complete, then change another tree trunk j as the new tracking target; repeat the above steps until the end of the video, and output the current number of fruits N .

Experimental platform and related settings

The platform for training the detection network included a desktop computer equipped with a CPU of Intel Core i5-6400 (2.70 GHz), 16 GB of RAM, and an NVIDIA GTX 1080 8 GB GPU, running on a Windows 10 64-bit system. The software tools included CUDA 9.0, cuDNN 7.1.3, Microsoft Visual Studio 2015, CMake-3.16, Python 3.6, and OpenCV 3.1.0. The experiments were implemented on the Darknet framework. The network input size was 416×416 , and 64 was for batch size. The stochastic gradient descent was applied for training with a momentum of 0.9 and a weight decay of 0.0005. An initial value of 0.001 and 50,000 were set as the learning rate and iterations of the network, respectively. Transfer learning technique was performed to train the network by importing weights from previous training on COCO dataset, which contains 91 objects types with 2.5 million labeled instances in 328k images (Lin et al., 2015).

The platform for debug the counting algorithm included a laptop computer equipped with a CPU of Intel Core i7-8565U (1.80 GHz), 8 GB of RAM, and an NVIDIA GeForce MX250 2 GB GPU, running on a Windows 10 64-bit system. The software tools included CUDA 10.0, cuDNN 7.6.5, Microsoft Visual Studio 2017, Python 3.8, and OpenCV 4.4.0. The algorithm was developed based on Python language with OpenCV library. The CSR-DCF algorithm and Darknet framework were integrated in the OpenCV library of 4.0 and later versions, which supports this research to realize the establishment of the CSRT tracker and the loading and detection of the YOLOv4-tiny network. 30 and 37 network layers were set as the YOLOv4-tiny network output layer, forward propagation to obtain the detection results of fruits and trunks. One of the detected trunks was selected as the tracking target to create a tracker for follow-up research.

Evaluation indicators

The performance of apple and trunk detection network trained in this study using both mean average precision (mAP) and average detection speed was assessed. mAP is calculated from the AP values of apple and trunk (Suo et al., 2021), which is defined by the following formulas.

$$P = \frac{TP}{TP+FP} \quad (5)$$

$$R = \frac{TP}{TP+FN} \quad (6)$$

$$AP = \int_0^1 P(R) dR \quad (7)$$

$$mAP = \frac{1}{2} (AP_{apple} + AP_{trunk}) \quad (8)$$

where P = precision

R = recall

TP = true positive

FN = false negative

FP = false positive

Another two metrics were proposed to evaluate the performance of apple counting in video frame, and P_c was defined in Eq. (11) to evaluate the performance of apple counting in the video.

$$MIDE = \frac{1}{n} \sum_{i=1}^n \frac{N_s}{N_c} \quad (9)$$

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (N_c - N_g)^2} \quad (10)$$

$$P_c = \left[1 - \frac{|P_t - N_t|}{N_t} \right] \times 100\% \quad (11)$$

where n = total video frames

N_s = the number of apple whose ID has changed compared to the previous frames

N_c = the number of counted apples in frame i using proposed algorithm

N_g = the average of manual counting by three different operators (ground truth) of frame I

P_t = the number of counted apples in a video using proposed algorithm

N_t = the average of manual counting by three different operators (ground truth) of video

$MIDE$ = mean ID calculation error

$RMSE$ = root mean square error

Results and discussion

In this section, the relationship between the information obtained by the algorithm and the real information measured on each video was evaluated and verified. Initially, the detection model of fruit and trunk trained based on YOLOv4-tiny network was evaluated according to test dataset with ground truth. Afterwards, the correctness of fruit IDs obtained based on proposed counting algorithm in each frame of the video were evaluated according to the fruit ID manually labeled. Then, a validation of the count of fruits in videos is presented. Finally, the shortcomings of this research and the future direction of the research are analyzed.

Performance of the detection network

The training loss curve based on YOLOv4-tiny on the training dataset converged within 50,000 iterations (Figure 4), and excellent detection results were obtained on the corresponding testing dataset (Table 1). The loss value is generally stable when the number of iterations reaches 43,000 iterations, and approached the lowest value of 0.90. The curve demonstrates that the network efficiently learns the features with good convergence ability, and the data in Table 2 is also sufficient to prove it. There are a total of 1280 images in testing dataset, including 37,642 targets. The AP of apples and trunks both reached more than 99%, and the mAP value was 99.35%, which provided a good basis for fruit counting. Besides, YOLOv4-tiny only took 0.022 s on average to detect apples or trunk in one image with the resolution of 1920×1080 pixels. The network trained in this work can quickly and effectively detect apples and trunks, which may achieve the desired effect in a real orchard.

The YOLOv4-tiny model achieved a good performance for detecting both apples and trunks under natural orchard (Table 1). Most apples were successfully detected from the background, although there were false negative (FN) and false positive (FP) detection boxes. However, the FN detection boxes would not make a significant negative effect on further apple counting because the fruit counting algorithm had great potential to correctly count the apple by detecting multiple video frames containing incorrectly classified apple (Figure 5). Because the trunks belong to the large target in orchard image and have obvious features, all trunks in the testing dataset are detected, even the incomplete trunks located at the boundary of the image. Although the detection rectangle of the tree trunk is not always the minimum bounding rectangle of the target, the fruit counting algorithm proposed in this paper can largely ignore the existence of this error.

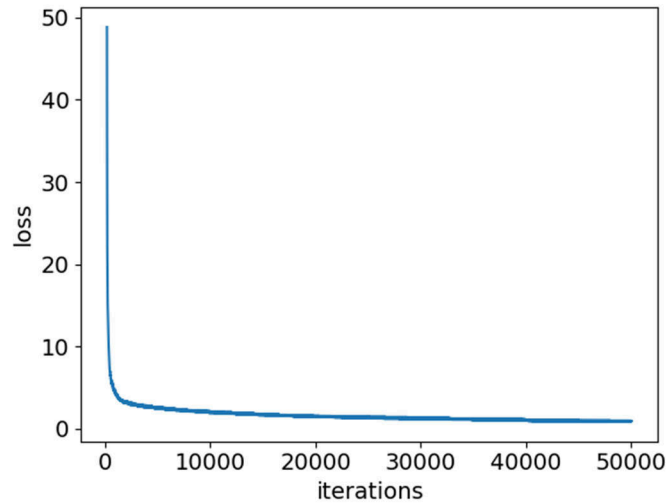


Figure 4. Training loss curve of YOLOv4-tiny network

Table 1. The detection results on the testing dataset

Object	TP	FP	FN	P/%	R/%	AP/%	mAP/%	Detection time/s
Apple	37376	4035	266	90.26	99.29	99.59	99.35	28
Trunk						99.10		

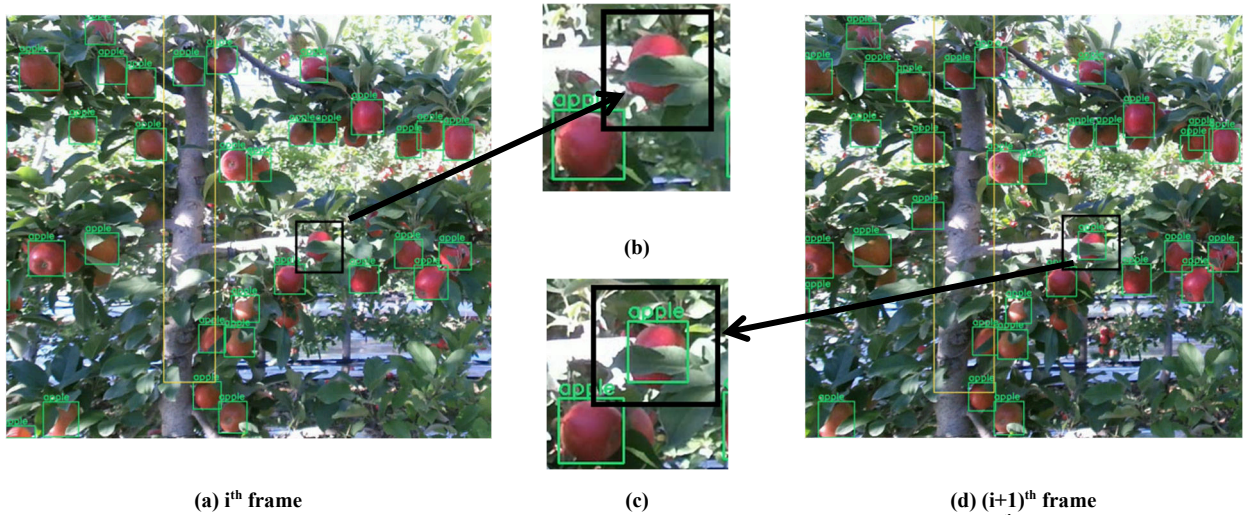


Figure 5. Examples of detection results of the same fruit in different video frames. (a) Fruit image detected in i^{th} frame of a video; (d) Fruit image detected in $(i+1)^{\text{th}}$ frame of the video. The detection results of the same fruit in the two video frames are marked with black rectangles and shown in (b) and (c), respectively.

Performance of the fruit counting algorithm

Figure 6 shows the overall performance of the method presenting the results of the linear regression. Each dot in the chart represents the maximum number of the fruit ID obtained in the last frame of a video found by the algorithm in the x axis (P_t) and the summation of the visual manual counts in the same video found by the inspectors in the y axis (N_t). This model fit was obtained from 15 orchard videos, which obtained a $R^2 = 0.9875$. The number of frames of these videos is between 10 and 104, which contains 76-478 fruits. Thus the model performed well, regardless of the number of video frames. P_c values of above orchard videos were all calculated (Figure 6), which reached an average value of 91.30%. For the selected videos, there was an average of 38 fruit detected per frame, of which about 0.01 false positive and 0.19 false negatives. The false positive errors resulted in an over-count while the false negative errors resulted in an under-count. The numbers of false positives and false negatives were similar, resulting in a low bias and a high P_c value.

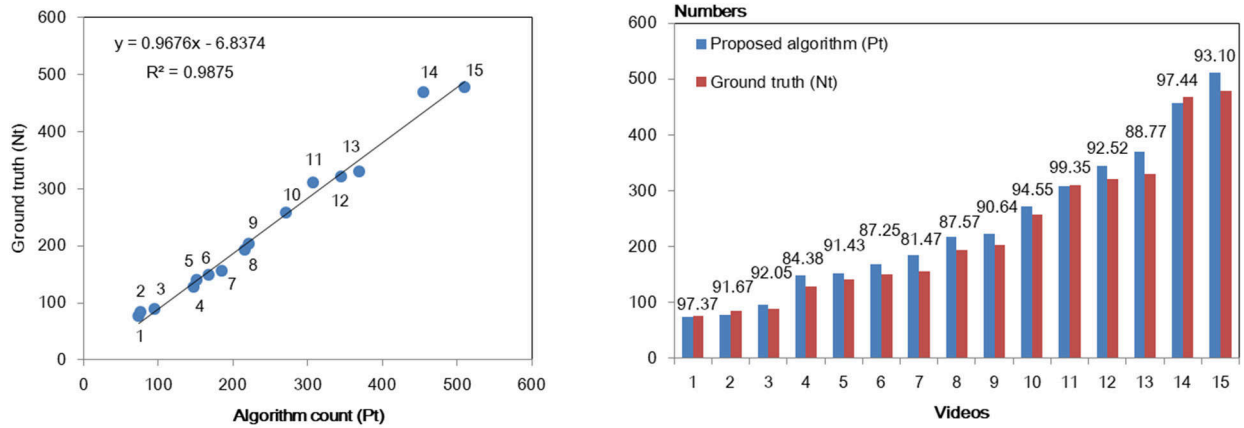


Figure 6. Fruit count results. On the left is a scatter plot of the Algorithm count against Ground truth count. The algorithm counts against the ground truth counts are shown on the right, as well as Pc values.

The counting algorithm proposed in this paper can give the correct ID to most fruits in the video frame. Figure 7 and Table 2 show 10 consecutive frames of a video and the N_s , N_c , and N_g values obtained by these frames, respectively. There are no fruit ID changes in multiple video frames, and at most 4 fruit IDs change in the same video frame. The change of fruit ID is the cause of the MIDE between video frames. N_c is greater than N_g in some video frames because the algorithm can detect the fruits that are occluded severely or located on the boundary in the frame, which is also one of the reasons for the emergence of RMSE. Besides, the unsuccessful detection of some fruits is another main reason for the emergence of RMSE. Based on Eq. (9) and Eq. (10), the relevant MIDE and RMSE are calculated to be 0.199 and 2.387, respectively, which indicates the reliability of the algorithm in short-term video fruit counting.

Among the factors that could interfere with the algorithm performance, major importance was given to the switch of fruit ID. The fruit ID changed mainly because the fruit was not successfully detected continuously between video frames. The system assigns a fruit ID when the fruit is successfully detected. If the fruit is successfully detected in subsequent frames, the original ID will remain unchanged; but when the fruit is not detected, the ID will be cancelled, regardless of whether the detection fails or the fruit field of view disappears. Therefore, the fruit that fails to be detected will be given another ID when it is successfully detected again, causing the ID of the fruit to change (Figure 8a). Besides, the close Euclidean distance between the fruits will also cause the fruit ID to change, provided that one of the two fruits has not been successfully detected in a certain frame (Figure 8b). However, the probability of this phenomenon occurring in the video is extremely low.

Table 2. The counting results on the 10 consecutive frames of a video

Indicators	Frame									
	37	38	39	40	41	42	43	44	45	46
N_s	0	0	0	0	2	0	1	0	4	1
N_c	33	37	38	38	37	37	37	37	43	40
N_g	37	35	36	38	38	40	40	39	40	41



Figure 8. Examples of the switch of fruit ID. (a) Failure to continuously successfully detect fruits between video frames causes the fruit ID switched; (b) The close Euclidean distance between the fruits causes the fruit ID switched

Comparison with other methods

Although there are some reports of apple fruit counts with good results, it is difficult to compare with this research because the data processed by each research were all captured images or videos from different orchard environments, and even the quality of captured orchard videos varies greatly. Vasconez et al. (2020) used a multi-object tracking method based on Gaussian estimation to simultaneously realize video counting of three fruits including apples, and reported that up to 93% of the fruit counting performance can be achieved. However, the highest detection rate of apple in this study was only 54%,

which means that there will be a large number of fruits that will not be counted in the study. Roy et al. (2019) proposed a method of combining multiple frame counts of videos based on camera motion, which can achieve a count accuracy of 89–98% on multiple datasets, but the research does not show that the processing fruit variety.

An apple counting algorithm based on the popular multi-target tracking algorithm ‘deep sort’ was developed to compare the performance of the algorithm proposed in this research. The basic idea of the deep Sort algorithm is tracking-by-detection. The core is the recursive Kalman filter and the Hungarian algorithm, which uses a metric that combines motion and appearance information to perform data association frame by frame. The algorithm detects the target in each frame of the video, and realizes the counting of the target by matching the detected target and the motion trajectory. In order to better compare the counting performance of deep sort and the algorithm proposed in this research, the YOLOv4-tiny network trained in this research is used to perform the detection work of deep sort algorithm. The same video was used to test the two algorithms.

Same videos are used to judge the performance of the two algorithms, which are shown in Figure 9a. The proposed algorithm has an average counting accuracy of 91.60% for these 5 videos, but the deep sort algorithm has serious over-counting. Although the deep sort algorithm has shown good tracking and counting performance for pedestrians, it is not ideal for apple video obtained in this study. This is because the fruit is not only much smaller than pedestrians or vehicles, and there is no obvious difference in appearance between individuals. Besides, although the researcher consciously controlled the average speed of the vehicle, the speed of the obtained video still varied greatly, making the movement in the fruit video basically irregular. This not only makes the metric that combines motion and appearance information basically unable to provide an effective function of fruit counting, but also leads to the failure of apple tracking, as shown in Figure 9b. In terms of speed, the algorithm proposed in this study can be implemented on CPU at a 2-5 fps near real-time system, but deep sort is only 0-2 fps.

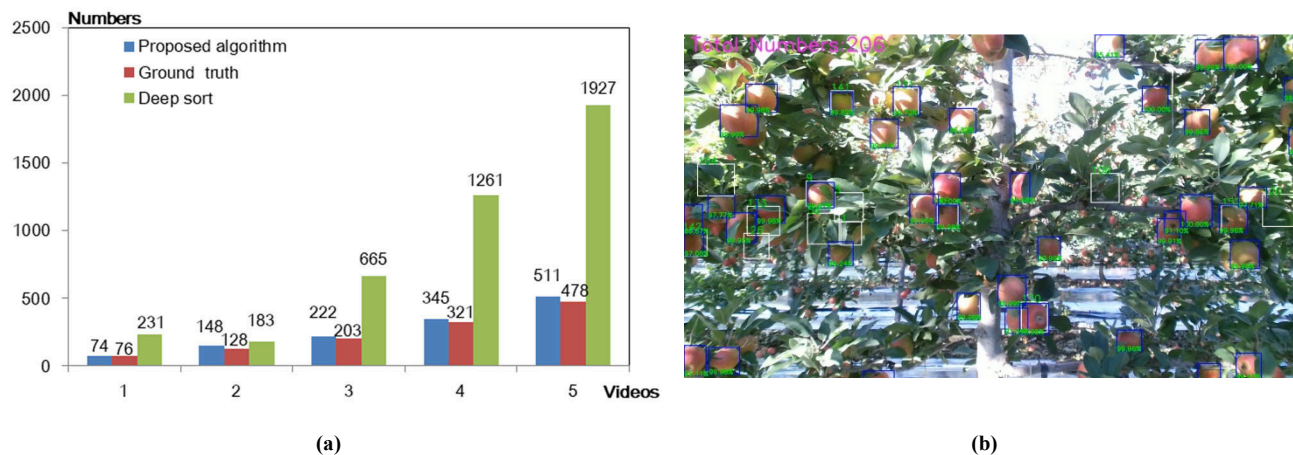


Figure 9. (a) Comparison of fruit count results based on 5 videos; (b) Example of tracking failure of deep sort algorithm. The rectangles of blue and white color are referring to the detected apples and tracked apples, respectively.

Future works

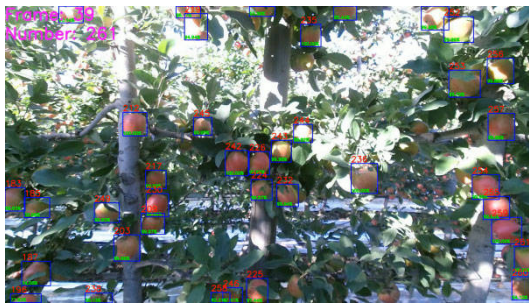
In the future, the fruit counts on both sides of the fruit tree row would to be merged to achieve an accurate estimation of orchard yield. In the modern orchard with vertical fruiting-wall architecture, some apples can be seen from both sides of the tree row, which results in repeated counting for those fruits when calculating the number of fruits in the tree row. Although there are some studies to reduce the existence of this error by establishing mathematical models(Chen et al., 2017; Koirala et al., 2019), this method can hardly cope with sudden changes in orchard yield. Roy et al. (2018) proposed a method for judging the visible fruits on both sides of a tree row based on three-dimensional reconstruction technology, which can help to achieve accurate estimation of orchard yield, but requires high configuration. Therefore, we plan to develop an algorithm that can identify the fruits visible on both sides of the tree row, combined with the current fruit counting algorithm to obtain the number of fruits in the orchard tree row in real-time. Besides, we also plan to develop technologies based on the depth camera to measure the diameter and volume of the fruit for accurate estimation of orchard yield.



(a) 37th frame



(b) 38th frame



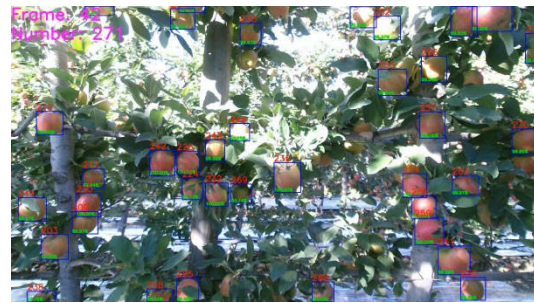
(c) 39th frame



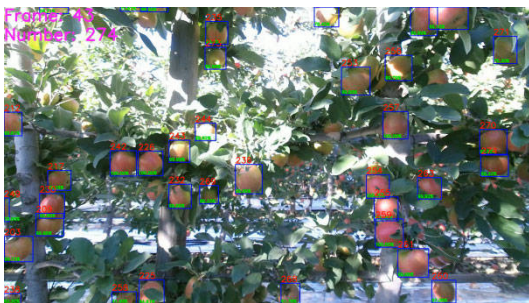
(d) 40th frame



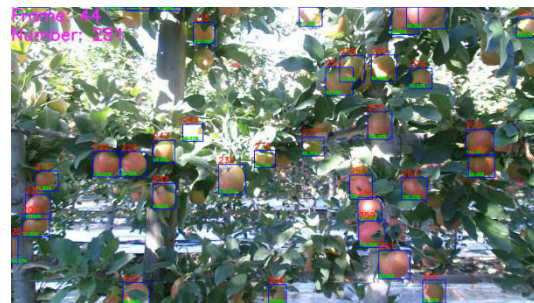
(e) 41th frame



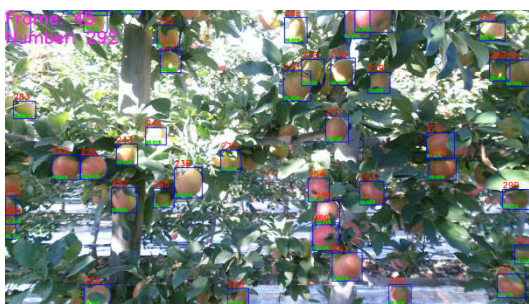
(f) 42th frame



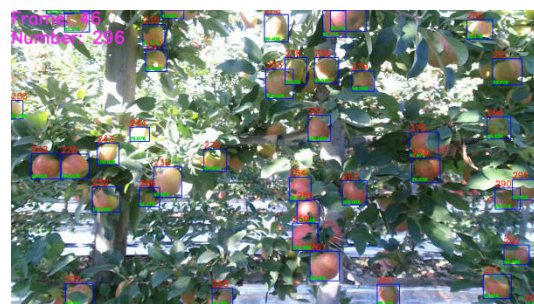
(g) 43th frame



(h) 44th frame



(i) 45th frame



(j) 46th frame

Figure 7 Example of detection and counting of fruits for 10 consecutive frames of video. The number of video frames and the number of fruits are displayed in the upper left corner; All detected fruits are marked by a blue rectangle; The number above the rectangle represents fruit ID.

Conclusion

This research examined automatically counting apples (during harvest season) on vertical-fruited-wall trees for estimation orchard yield using algorithm developed by YOLOv4-tiny and CSRT in orchard videos. A total of 800 images were acquired and then augmented to 6400 images, where 80% of them were used to train the YOLOv4-tiny model. Both apples and trunks were labeled as study targets, reaching AP values of 99.59% and 99.10%, respectively. The detected trunk was used as the tracking target in CSRT tracker to obtain the reference displacement between video frames. Apples detected between adjacent video frames are combined with reference displacement information to achieve matching of the same fruit. Assign the same fruit unique ID to complete the fruit count. The algorithm counts fruit videos in a 2-5 fps real-time system, and achieved a counting accuracy of 91.30%, which provides the possibility for real-time yield estimation of the orchard.

Acknowledgements

This work was supported by the China Postdoctoral Science Foundation funded project (2019M663832); Fundamental Research Funds for the Central Universities of China (2452020170); National Natural Science Foundation of China (grant number 31971805); Recruitment Program of High-End Foreign Experts of the State Administration of Foreign Experts Affairs, Ministry of Science and Technology, China (grant number G20200027075).

References

- Apolo-Apolo, O.E., Martínez-Guanter, J., Egea, G., Raja, P., Pérez-Ruiz, M., (2020). Deep learning techniques for estimation of the yield and size of citrus fruits using a UAV. *Eur. J. Agron.*, 115, 126030. <https://doi.org/10.1016/j.eja.2020.126030>
- Bargoti, S., Underwood, J.P., (2017). Image segmentation for fruit detection and yield estimation in apple orchards. *J. F. Robot.*, 34(6), 1039–1060. <https://doi.org/10.1002/rob.21699>
- Bochkovskiy, A., Wang, C.-Y., Liao, H.-Y.M., (2020). YOLOv4: Optimal speed and accuracy of object detection. <https://arxiv.org/abs/2004.10934v1>. Accessed 12 May 2021.
- Chen, S.W., Shivakumar, S.S., Dcunha, S., Das, J., Okon, E., Qu, C., Taylor, C.J., Kumar, V., (2017). Counting apples and oranges with deep learning: A data-driven approach. *IEEE Robot. Autom. Lett.*, 2, 781–788. <https://doi.org/10.1109/LRA.2017.2651944>
- Danelljan, M., Häger, G., Khan, F.S., Felsberg, M., (2014). Accurate scale estimation for robust visual tracking, *Proc. Br. Mach. Vis. Conf. 2014*. <https://doi.org/10.5244/C.28.65>
- Danelljan, M., Robinson, A., Khan, F.S., Felsberg, M., (2016). Beyond correlation filters: Learning continuous convolution operators for visual tracking, *Lect. Notes Comput. Sci. (including Subser. Lect. Notes Artif. Intell. Lect. Notes Bioinformatics)*, 9909 LNCS, 472–488. https://doi.org/10.1007/978-3-319-46454-1_29
- Dorj, U.O., Lee, M., Yun, S. seok, (2017). An yield estimation in citrus orchards via fruit detection and counting using image processing. *Comput. Electron. Agric.*, 140, 103–112. <https://doi.org/10.1016/j.compag.2017.05.019>
- Farhodov, X., Kwon, O.-H., Moon, K.-S., Kwon, O., Lee, S.-H., Kwon, K.-R., (2019). A new CSR-DCF tracking algorithm based on Faster RCNN detection model and CSRT tracker for drone data. *J. Korea Multimed. Soc.*, 22(12), 1415–1429. <https://doi.org/10.9717/kmms.2019.22.12.1415>
- Fu, L., Feng, Y., Majeed, Y., Zhang, X., Zhang, J., Karkee, M., Zhang, Q., (2018). Kiwifruit detection in field images using Faster R-CNN with ZFNet. *IFAC-PapersOnLine*, 51(17), 45–50. <https://doi.org/10.1016/j.ifacol.2018.08.059>
- Fu, L., Feng, Y., Wu, J., Liu, Z., Gao, F., Majeed, Y., Al-Mallahi, A., Zhang, Q., Li, R., Cui, Y., (2020a). Fast and accurate detection of kiwifruit in orchard using improved YOLOv3-tiny model. *Precis. Agric.*, 22, 754–776. <https://doi.org/10.1007/s11119-020-09754-y>
- Fu, L., Majeed, Y., Zhang, X., Karkee, M., Zhang, Q., (2020b). Faster R-CNN-based apple detection in dense-foliage fruited-wall trees using RGB and depth features for robotic harvesting. *Biosyst. Eng.*, 197, 245–256. <https://doi.org/10.1016/j.biosystemseng.2020.07.007>
- Gao, F., Fu, L., Zhang, X., Majeed, Y., Li, R., Karkee, M., Zhang, Q., (2020). Multi-class fruit-on-plant detection for apple in SNAP system using Faster R-CNN. *Comput. Electron. Agric.*, 176, 105634. <https://doi.org/10.1016/j.compag.2020.105634>
- Gené-Mola, J., Vilaplana, V., Rosell-Polo, J.R., Morros, J.R., Ruiz-Hidalgo, J., Gregorio, E., (2019). Multi-modal deep learning for Fuji apple detection using RGB-D cameras and their radiometric capabilities. *Comput. Electron. Agric.*, 162, 689–698. <https://doi.org/10.1016/j.compag.2019.05.016>
- Han, B.G., Lee, J.G., Lim, K.T., Choi, D.H., (2020). Design of a scalable and fast yolo for edge-computing devices. *Sensors*

- (Switzerland), 20(23), 1–15. <https://doi.org/10.3390/s20236779>
- Hare, S., Saffari, A., Torr, P.H.S., (2011). Struck: Structured output tracking with kernels, *Proc. IEEE Int. Conf. Comput. Vis.*, pp. 263–270. Barcelona: IEEE. <https://doi.org/10.1109/ICCV.2011.6126251>
- Henriques, J.F., Caseiro, R., Martins, P., Batista, J., (2015). High-speed tracking with kernelized correlation filters. *IEEE Trans. Pattern Anal. Mach. Intell.*, 37, 583–596. <https://doi.org/10.1109/TPAMI.2014.2345390>
- Jain, M., Gemert, J. Van, Bouthemy, P., Snoek, C.G.M., (2014). Action localization with tubelets from motion, *2014 IEEE Conf. Comput. Vis. Pattern Recognit.*, pp. 740–747. Colombia: IEEE. <https://doi.org/10.1109/CVPR.2014.100>
- Kalal, Z., Mikolajczyk, K., Matas, J., (2012). Tracking-learning-detection. *IEEE Trans. Pattern Anal. Mach. Intell.*, 34(7), 1409–1422. <https://doi.org/10.1109/TPAMI.2011.239>
- Koirala, A., Walsh, K.B., Wang, Z., McCarthy, C.L., (2019). Deep learning for real-time fruit detection and orchard fruit load estimation: benchmarking of ‘MangoYOLO.’ *Precis. Agric.*, 20(6), 1107–1135. <https://doi.org/10.1007/s11119-019-09642-0>
- Kuznetsova, A., Maleva, T., Soloviev, V., (2020). Using YOLOv3 algorithm with pre- And post-processing for apple detection in fruit-harvesting robot. *Agronomy*, 10(7), 1016. <https://doi.org/10.3390/agronomy10071016>
- Lee, H., Cho, A., Lee, S., Whang, M., (2019). Vision-Based Measurement of Heart Rate from Ballistocardiographic Head Movements Using Unsupervised Clustering. *Sensors*, 19, 3263. <https://doi.org/10.1201/9781315382586-12>
- Li, B., Wu, T., Zhu, S.C., (2014). Integrating context and occlusion for car detection by hierarchical and-or model. *Lect. Notes Comput. Sci. (including Subser. Lect. Notes Artif. Intell. Lect. Notes Bioinformatics)*, 8694 LNCS, 652–667. https://doi.org/10.1007/978-3-319-10599-4_42
- Lin, T., Maire, M., Belongie, S., Bourdev, L., Girshick, R., Hays, J., Perona, P., Ramanan, D., Zitnick, C.L., Doll’ar, P., (2015). Microsoft COCO : Common objects in context. *Lect. Notes Comput. Sci. (including Subser. Lect. Notes Artif. Intell. Lect. Notes Bioinformatics)*, 8693 LNCS, 740–755. https://doi.org/10.1007/978-3-319-10602-1_48
- Liu, X., Chen, S.W., Liu, C., Shivakumar, S.S., Das, J., Taylor, C.J., Underwood, J., Kumar, V., (2019a). Monocular camera based fruit counting and mapping with semantic data association. *IEEE Robot. Autom. Lett.*, 4(3), 2296–2303. <https://doi.org/10.1109/LRA.2019.2901987>
- Liu, X., Zhao, D., Jia, W., Ji, W., Sun, Y., (2019b). A detection method for apple fruits based on color and shape features. *IEEE Access*, 7, 67923–67933. <https://doi.org/10.1109/ACCESS.2019.2918313>
- Lukežič, A., Vojir, T., Čehovin Zajc, L., Matas, J., Kristan, M., (2018). Discriminative Correlation Filter Tracker with Channel and Spatial Reliability. *Int. J. Comput. Vis.*, 126(7), 671–688. <https://doi.org/10.1007/s11263-017-1061-3>
- Milan, A., Rezatofighi, S.H., Dick, A., Reid, I., Schindler, K., (2017). Online multi-target tracking using recurrent neural networks. <https://arxiv.org/abs/1604.03635>. Accessed 12 May 2021.
- Montalbo, F.J.P., (2020). A computer-aided diagnosis of brain tumors using a fine-tuned yolo-based model with transfer learning. *KSII Trans. Internet Inf. Syst.*, 14(12), 4816–4834. <https://doi.org/10.3837/tiis.2020.12.011>
- Ngo, T.N., Wu, K.C., Yang, E.C., Lin, T. Te, (2019). A real-time imaging system for multiple honey bee tracking and activity monitoring. *Comput. Electron. Agric.*, 163, 104841. <https://doi.org/10.1016/j.compag.2019.05.050>
- Payne, A., Walsh, K., Subedi, P., Jarvis, D., (2014). Estimating mango crop yield using image analysis using fruit at “stone hardening” stage and night time imaging. *Comput. Electron. Agric.*, 100, 160–167. <https://doi.org/10.1016/j.compag.2013.11.011>
- Qureshi, W.S., Payne, A., Walsh, K.B., Linker, R., Cohen, O., Dailey, M.N., (2017). Machine vision for counting fruit on mango tree canopies. *Precis. Agric.*, 18(2), 224–244. <https://doi.org/10.1007/s11119-016-9458-5>
- Redmon, J., Divvala, S., Girshick, R., Farhadi, A., (2016). You only look once: Unified, real-time object detection, *Proc. IEEE Comput. Soc. Conf. Comput. Vis. Pattern Recognit.*, pp. 779–788. Las Vegas: IEEE Computer Society. <https://doi.org/10.1109/CVPR.2016.91>
- Redmon, J., Farhadi, A., (2018). YOLOv3: An incremental improvement. <https://arxiv.org/abs/1804.02767v1>. Accessed 12 May 2021.
- Redmon, J., Farhadi, A., (2017). YOLO9000: Better, faster, stronger. *Proc. - 30th IEEE Conf. Comput. Vis. Pattern Recognition, CVPR 2017-Janua*, 6517–6525. <https://doi.org/10.1109/CVPR.2017.690>
- Roy, P., Dong, W., Isler, V., (2018). Registering Reconstructions of the Two Sides of Fruit Tree Rows, *IEEE Int. Conf. Intell. Robot. Syst.*, pp. 7697–7702, Madrid: IEEE. <https://doi.org/10.1109/IROS.2018.8594167>
- Roy, P., Kislay, A., Plonski, P.A., Luby, J., Isler, V., (2019). Vision-based preharvest yield mapping for apple orchards. *Comput. Electron. Agric.*, 164, 104897. <https://doi.org/10.1016/j.compag.2019.104897>
- Song, Z., Zhou, Z., Wang, W., Gao, F., Fu, L., Li, R., Cui, Y., (2021). Canopy segmentation and wire reconstruction for kiwifruit robotic harvesting. *Comput. Electron. Agric.*, 181, 105933. <https://doi.org/10.1016/j.compag.2020.105933>
- Stein, M., Bargoti, S., Underwood, J., (2016). Image based mango fruit detection, localisation and yield estimation using

- multiple view geometry. *Sensors*, 16, 1915. <https://doi.org/10.3390/s16111915>
- Suo, R., Gao, F., Zhou, Z., Fu, L., Song, Z., Dhupia, J., Li, R., Cui, Y., (2021). Improved multi-classes kiwifruit detection in orchard to avoid collisions during robotic picking. *Comput. Electron. Agric.*, 182, 106052. <https://doi.org/10.1016/j.compag.2021.106052>
- Vasconez, J.P., Delpiano, J., Vougioukas, S., Auat Cheein, F., (2020). Comparison of convolutional neural networks in fruit detection and counting: A comprehensive evaluation. *Comput. Electron. Agric.*, 173, 105348. <https://doi.org/10.1016/j.compag.2020.105348>
- Wang, C.-Y., Bochkovskiy, A., Liao, H.-Y.M., (2020). Scaled-YOLOv4: Scaling cross stage partial network. <https://arxiv.org/abs/2004.10934v1>. Accessed 12 May 2021
- Wang, C., (2018). Detection and counting of immature green citrus fruit based on the Local Binary Patterns (LBP) feature using illumination-normalized images. *Precis. Agric.*, 19, 1062–1083. <https://doi.org/10.1007/s11119-018-9574-5>
- Wang, Q., Zhang, L., Bertinetto, L., Hu, W., Torr, P.H.S., (2019). Fast online object tracking and segmentation: A unifying approach, *Proc. IEEE Comput. Soc. Conf. Comput. Vis. Pattern Recognit.*, pp. 1328–1338. Long Beach: IEEE Computer Society. <https://doi.org/10.1109/CVPR.2019.00142>
- Wang, Z., Walsh, K., Koirala, A., (2019). Mango fruit load estimation using a video based MangoYOLO—Kalman filter—hungarian algorithm method. *Sensors (Switzerland)*, 19(12), 2742. <https://doi.org/10.3390/s19122742>
- Wawrzyniak, N., Hyla, T., Popik, A., (2019). Vessel detection and tracking method based on video surveillance. *Sensors (Switzerland)*, 19(23), 5230. <https://doi.org/10.3390/s19235230>
- Wojke, N., Bewley, A., Paulus, D., (2018). Simple online and realtime tracking with a deep association metric. <https://arxiv.org/abs/1604.03635>. Accessed 12 May 2021.
- Zhang, X., Fu, L., Majeed, Y., He, L., Karkee, M., Whiting, M.D., Zhang, Q., 2018. Field Evaluation of Data-based Pruning Severity Levels (PSL) on Mechanical Harvesting of Apples. *IFAC-PapersOnLine* 51, 477–482. <https://doi.org/10.1016/j.ifacol.2018.08.164>
- Zhou, Z., Song, Z., Fu, L., Gao, F., Li, R., Cui, Y., 2020. Real-time kiwifruit detection in orchard using deep learning on AndroidTM.pdf. *Comput. Electron. Agric.* 179, 105856. <https://doi.org/10.1016/j.compag.2020.105856>